Computer Graphics: Lecture 9

The Matrix

Welcome back!

Last time we reviewed the math we all forgot:

- We looked at the trig functions…
- …and their inverses.
- The pythagorean theorem
- The dot product
- The cross product
- We saw some examples of when those things are handy in actual games or simulations.

This time

This is the most important mathematical lesson we're going to have.

We're going to learn about matrices.

But we're not just going to review: we're going to show how they enable 3D operations.

The matrix

So, okay. A matrix is a grid of numbers. Here's one:

$$\begin{pmatrix} 1 & 0 & 0 \\ 0 & 2 & 0 \\ 0 & 0 & 3 \end{pmatrix}$$

But what does a matrix actually represent, and what does the number of rows and columns tell us about it?

Matrices are linear transformations

A matrix represents a function from vectors to vectors.

Specifically, they encode a linear function, where each componenent of the output is a **linear combination** of inputs.

What's a **linear combination**? It's a weighted sum.

So if we apply the function, each of the output values is treated as a weighted sum of the input values.

The rows of the matrix supply the weights.

Example matrix

Consider this simple 2 by 2 matrix: $\begin{pmatrix} 1 & 2 \\ 3 & 4 \end{pmatrix}$

This matrix encodes a function: f(v) = v', where `v'.x = v.x + 2*v.y` and `v'.y = 3*v.x + 4*v.y`.

Those are both weighted sums.

To *call* the function represented by a matrix, we post-multiply it by the column matrix we want to pass as input.

Applying a matrix

For example, let's call that matrix on the vector `<5, 6>`

$$\begin{pmatrix} 1 & 2 \\ 3 & 4 \end{pmatrix} \begin{pmatrix} 5 \\ 6 \end{pmatrix} = \begin{pmatrix} 5 \cdot 1 + 6 \cdot 2 \\ 5 \cdot 3 + 6 \cdot 4 \end{pmatrix} = \begin{pmatrix} 17 \\ 39 \end{pmatrix}$$

The columns provide a particularly good way to understand a matrix. Each column of a matrix is a **basis vector**. Each **basis vector** tells you how the matrix will transform the corresponding input component.

The basis vector `<1, 3>` above tells us that for every unit of `x` input, the output will move in the `<1, 3>` direction by one unit. So we moved 5 units in that direction, and 6 units in the `<2, 4>` direction.

Why bother?

If you've tried to learn 3D graphics before, you probably were given a list of matrices. It's near the beginning of any 3D curriculum.

Why? Because there are a lot of things we want to do to our models: stretch them, squeeze them, rotate them, move them around, look at them in a camera, project the camera into perspective.

All of these things can be done with matrices (assuming we're in a projective space, which I will talk about later).

Why bother? (2)

But more importantly: those things can all be done *in sequence* with a single matrix. One matrix can represent *every sequence of any one* of these actions.

We can stretch, squeeze, rotate, move, view, and project, in one single operation.

In fact, we can stretch three times, then move, then stretch again.

That sequence of actions will have *one* corresponding matrix.

Why bother? (3)

Therefore, it's wonderful for computer graphics: we don't need to submit a list of actions to the video card. A single matrix suffices.

Granted, there is more than one *kind* of thing that needs to be transformed. In practice, there will usually be several matrices that the video card sees:

- One for the model (stretching, squashing, moving, rotating)
- One for the camera (viewing)
- One for the projection (perspective, can be combined with camera)
- One for each animation bone (when using skeletal animations)
- Lights can use matrices, too.

Questions?

Matrix sizes

A matrix does not have to be square.

The number of columns of a matrix tells you how many components the input vector must have.

The number of rows tells you have many ouput components the matrix will have.

If a matrix has fewer rows than columns, the matrix is necessarily cutting away dimensions.

Likewise, if a matrix has more rows than columns, the matrix is necessarily adding dimensions.

Matrix sizes (2)

However, our matrices will be square. A square matrix results in the same number of components of output as input.

Since we're working in 3D, let's start with a 3x3 matrix and see how far that gets us.

Each matrix will represent an operation that we want to do, such as rotation. To use the matrix, we will multiply it by all the vertex positions in the model.

Assume the model will be centered, and that these operations happen around the origin.

The scaling matrix

One of the simplest matrices is the scaling matrix. It can be used to stretch and squeeze a 3D model when we apply it to the vertices.

$$\begin{pmatrix} 1 & 0 & 0 \\ 0 & 2 & 0 \\ 0 & 0 & 3 \end{pmatrix}$$

This matrix will keep the x component the same. The y component will double, and the z component will triple.

So if we applied this matrix to `<2, 2, 2>` we would get `<2, 4, 6>`

All the vertices of the model are pushed away (or pulled toward for values less than 1) zero.

The identity matrix

If we “scale” every component by 1, then nothing will change. This is the identity matrix. It is the matrix equivalent of 1. Multiply a compatible vector by it: nothing happens.

$$\begin{pmatrix} 1 & 0 & 0 \\ 0 & 1 & 0 \\ 0 & 0 & 1 \end{pmatrix}$$

When is this one useful? It’s very useful for debugging: set the current model’s matrix to the identity and it should be hanging out the origin, looking the same as it looked in the 3D modelling program.

The rotation matrix

This is the first one that requires some explanation.

Let's start by figuring out how it works in 2D.

If we want to rotate a *point* in 2D. How do we do it?

E.g., let's say we have the point described by the vector `<1, 1>`. How do we rotate that by, say, 10% of a turn around the origin?

Rotation in 2D

Let's break it down by component.

Imagine that we rotated just the x axis.

The new value of the axis was pointing at (1, 0). Now it will point to $(\cos(\theta), \sin(\theta))$.

The new y axis is going to be rotated the same amount, but 25%τ forward: (cos(theta + 25%τ), sin(theta + 25%τ)

This results in $-\sin(\theta), \cos(\theta)$ to be the new y-axis.

But what do we do with these new axes?

Rotation in 2D (2)

We take the old x value and multiply it by the new x-axis. That stretched x axis is part of the answer.

We take the old y value and multiply it by the new y-axis. That stretched y axis is the other part of our answer.

We add those two parts together.

I've just described matrix multiplication by this matrix:

$$\begin{pmatrix} \cos(\theta) & -\sin(\theta) \\ \sin(\theta) & \cos(\theta) \end{pmatrix}$$

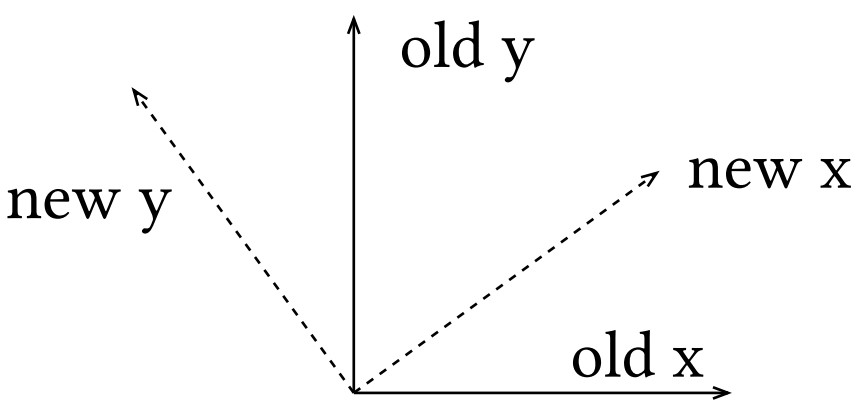
Rotation in 2D–basis vector version

But it's easier to understand if we think about it as basis vectors.

A 2x2 matrix is just two independent axes: one x, one y.

Rotate *both* of them by theta.

The new matrix just has the basis vectors pointing in the rotated direction:



Rotation about Z in 3D

It turns out, if we want to rotate within the X-Y plane like that, in three dimensions it's exactly the same, except we ignore the z axis. That is, the z axis stays the same:

$$\begin{pmatrix} \cos\theta & -\sin\theta & 0 \\ \sin\theta & \cos\theta & 0 \\ 0 & 0 & 1 \end{pmatrix}$$

The rotation matrix around the Z-axis.

But what if we want to rotate around the X or Y axes?

Rotation about X or Y in 3D

The axis we're rotating around doesn't change. It gets "passed through".

Here is rotation around X:

$$\begin{pmatrix} 1 & 0 & 0 \\ 0 & \cos\theta & -\sin\theta \\ 0 & \sin\theta & \cos\theta \end{pmatrix}$$

Here is rotation around Y[1]:

$$\begin{pmatrix} \cos\theta & 0 & \sin\theta \\ 0 & 1 & 0 \\ -\sin\theta & 0 & \cos\theta \end{pmatrix}$$

[1]The signs on the sines are flipped because Z comes *out* of the screen, so the relationship between Z and X are opposite the relationship between X and Y.

Handedness of coordinate systems

Just a quick aside: I mentioned that the Z-coordinate comes *out* of the screen, and the relationship between Z and X is what caused the signs to be flipped on some of the coefficients.

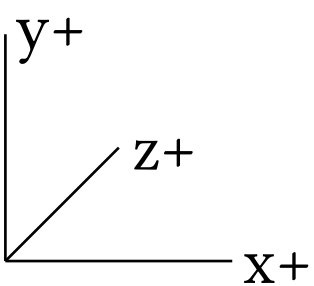
Z doesn't have to come out of the screen. If it does, we say that the coordinate system is **right-handed**.

If Z goes *into* the screen, the coordinate system is **left-handed**.

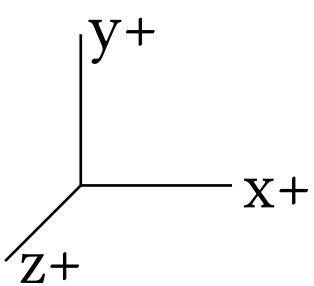
Handedness of coordinate systems (2)

Hold your left hand up and make an “L”. Poke your middle finger forward. Your middle finger is “+Z”, your thumb is “+X”, and your index finger is “+Y”.

Hold your right hand up with your thumb and index finger going the same way. You will have to twist it around. Z (your middle finger) will point toward you.



Left handed



Right handed

Handedness of coordinate systems (3)

The depth buffer in WebGPU is actually left-handed: larger Z values are considered behind smaller ones, so Z goes into the screen.

In the old days, OpenGL's matrix helper library was right-handed. DirectX used left-handed matrices.

Right handed is what math textbooks typically use, and it seems like it ended up winning out. Now, DirectX's math library supports both.

The matrix library we are going to use, `gl-Matrix`, uses right-handed perspective matrices. However, before we get to perspective, we will use left-handed coordinates (which is how the hardware works naturally).

Shearing

One more linear transformation: shearing.

I won't go into a lot of detail because shearing is the most complex transform, and also the one we use the least (with one exception).

The basic idea is to move one of the basis vectors by a fixed amount:

$$\begin{pmatrix} 1 & 0 & 0 \\ d & 1 & 0 \\ f & 0 & 1 \end{pmatrix}$$

A shear matrix

It's mathematically useful (rotations can be decomposed into shears), but it's not an operation that we frequently do in 3D computer graphics.

Questions?

The missing operation

Keep in mind, these operations are supposed to include *every positional transformation* we want to do to a 3D model.

There's one missing: translation.

How can we do translation in 3D?

$$\begin{pmatrix} ? & ? & ? \\ ? & ? & ? \\ ? & ? & ? \end{pmatrix}$$

[class?]

Translation

That was a trick question: we can't.

At least, we can't do translation in 3D with a 3-by-3 matrix.

We can add a vector to translate it, but we can't express that as a linear transformation.

So what? The problem is: we want to be able to concatenate all our transformations into one matrix. If we have random translations, suddenly we have to store our operations in a list or something.

We *can* do translation in 3D with a 4-by-4 matrix, though, if we change our coordinate system.

Homogenous Coordinates

Homogenous coordinates allow us to use the same matrices to transform both points and vectors.

Normally, matrices transform vectors. But vectors don't have a position.

Linear transformation matrices assume that the origin is 0.

This won't work. We want to be able to *move* the origin by doing a matrix multiplication.

Homogenous coordinates (2)

When we use homogenous coordinates, we add a dimension, named ‘w’.

The `w` dimension describes whether a value is a vector or a point, and if it’s a point, how much it needs to be scaled.

If the `w` dimension is 0, the value is a vector. `<1, 2, 0>` is basically the same vector as `<1, 2>`.

If the `w` dimension is 1, the value is a point. `<1, 2, 1>` is a point floating in space.

If the `w` dimension is neither 0 nor 1, then it represents the same point as if we divided by `w`. So `<2, 4, 2>` `<3, 6, 3>` are the same as `<1, 2, 1>`

Visualizing homogenous coordinates

With homogeneous coordinates, we *project* points onto the plane w = 1.

The w axis points away from the origin.

What makes x and y shrink as they get closer to the origin?

Answer: we divide by w.

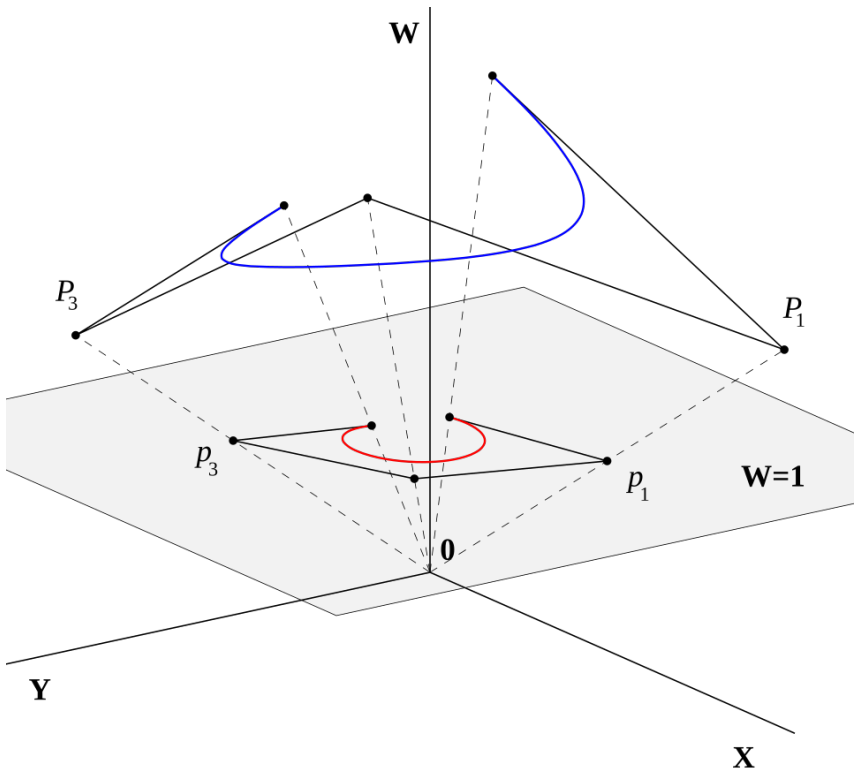


Image by Wojciech Muła, CC-BY-SA 3.0

Homogenous coordinates (3)

By dividing by w, we normalize the homogenous coordinates.

Your GPU does this automatically after the vertex shader runs. It's called the w divide.

This is how perspective can be achieved on a GPU. We'll learn how to enable perspective next time.

These homogenous coordinates are the reason why, most of the time, we use 4-by-4 transformation matrices instead of 3-by-3 ones.

So what does the translation matrix look like?

A translation matrix

$$\begin{pmatrix} 1 & 0 & 0 & t_x \\ 0 & 1 & 0 & t_y \\ 0 & 0 & 1 & t_z \\ 0 & 0 & 0 & 1 \end{pmatrix}$$

The translation matrix

Consider what would happen if we performed this multiplication:

$$\begin{pmatrix} 1 & 0 & 0 & 1 \\ 0 & 1 & 0 & 2 \\ 0 & 0 & 1 & 3 \\ 0 & 0 & 0 & 1 \end{pmatrix} \begin{pmatrix} 2 \\ 3 \\ 4 \\ 1 \end{pmatrix} = ???$$

A translation matrix (2)

Let's work the math:

$$\begin{pmatrix} 1 & 0 & 0 & 1 \\ 0 & 1 & 0 & 2 \\ 0 & 0 & 1 & 3 \\ 0 & 0 & 0 & 1 \end{pmatrix} \begin{pmatrix} 2 \\ 3 \\ 4 \\ 1 \end{pmatrix} = \begin{pmatrix} 2{\cdot}1+0+0+1{\cdot}1 \\ 0+3{\cdot}1+0+1{\cdot}2 \\ 0+0+4{\cdot}1+1{\cdot}3 \\ 0+0+0+1{\cdot}1 \end{pmatrix} = \begin{pmatrix} 3 \\ 5 \\ 7 \\ 1 \end{pmatrix}$$

Notice: because there's a 1 in the w-place of the vector, we just add the w-column of the matrix to the result. This lets us achieve translation with a *multiplication* rather than addition.

If there had been a 2 in the w-place, we would have translated twice as much. The result would be the same after dividing by w.

If there had been a 0 in the w-place, we wouldn't have translated at all.

Affine transformations

All the transformations we have seen today are **affine** transformations.

An affine transformation is a linear transformation that can also optionally move the origin.

We will use these matrices to position our 3D models in our scene. We can rotate, translate, scale, even shear them however we want.

We take these matrices and we *multiply* them.

The resulting matrix *combines* all the transformations.

Order of matrix operations

What I'm about to show you is tricky and is something that people typically get wrong a lot.

Suppose we want to first, translate by some vector (T), then rotate (R).

Do we represent the compound transformation as:

- TR, or
- RT?

Note: these are two different operations! If we rotate first, we rotate in place, then move. If we translate first, we rotate around the origin and will end up orbiting around the origin by the angle.

Order of matrix operations (2)

The correct operation order is…`RT`

That's right. It's the opposite of what you expect.

Remember that matrices represent functions. Suppose R and T were functions, and p was our point. We would write this: R(T(p))

That is, T would happen first, but because the name of the function comes first, the letter T appears after R.

Matrices work the same way!

Order of matrix operations (3)

In general, if A, B, and C are matrices:

ABC means *first do C*, then B, then A

If you want to actually go in the order of first A, then B, then C: `CBA`.

This is confusing, and it will catch you many times. The solution is to think of a matrix as a function.

Associativity but not commutativity

Remember that matrix operations are associative, but not necessarily commutative.

That means, we can pre-multiply matrices if we want, but we can't flip the left and right operands.

For example, `A(BC)` is the same as `(AB)C`.

But `AB` is not usually the same as `BA`.

Let's take a quick question break, then learn how to install the matrix library we're going to use

Questions

Installing `gl-matrix`

In your project directory, run: `npm install gl-matrix`

Now it's installed, but there are more things you have to do.

First, we want to make sure Typescript understands it. In our `tsconfig.json` file add this key and these values:

```
"paths": {
      "gl-matrix": ["../node_modules/gl-matrix/esm/index.js"],
      "gl-matrix/*": ["./node_modules/gl-matrix/esm/*.js"]
},
```

Installing `gl-matrix` (2)

Lastly, we have store a source map in our HTML file.

This makes it so that `import` statements in the javascript get resolved to the right place.

Here is mine. This goes above the `<script....>` tag:

```
<script type="importmap">
{
    "imports": {
        "gl-matrix": "./node_modules/gl-matrix/esm/index.js",
        "gl-matrix/": "./node_modules/gl-matrix/esm/"
    }
}
</script>
```

Installing `gl-matrix` (3)

This tells the browser that when `gl-matrix/something` is imported, to load `./node_modules/gl-matrix/esm/something`

If `gl-matrix` is imported by itself, it will load its `index.js`.

Make sure that the path exists for you.

If you didn't put node modules in the same directory as your HTML file, it won't.

Remember, '.' means the current directory. So './node_modules/...' means 'look in the current directory for a directory called `node_modules`, and then look under that ...

Making a matrix

You'll want to read the documentation to make sure you understand the basic operations are available, but let's show off some simple matrices.

There are 3 basic matrix types: `mat2`, `mat3` and `mat4`. We'll mainly be using `mat4` because it supports all the affine matrices we need.

To create an identity matrix, run the static method `mat4.create()`

This is a **static method**. It's like a regular function inside of the `mat4` class, not a method. You don't call it on a specific matrix, you call it on the class `mat4`.

Most of the matrix functions are like this.

Making a matrix (2)

The gl-matrix library wants to be fast.

For that reason, we usually only have to `create` a matrix once.

There is a method for each affine matrix. It will take an existing matrix, apply the transformation, and *write the result into another matrix*.

This allows you to avoid unecessary allocations.[1]

[1] although typescript doesn't really know this, so I frequently use `mat4.create()` so that it knows the result is a matrix.

Making a matrix (3)

Let's make a simple matrix that will first rotate, then translate:

```
const matTranslate = mat4.create();
mat4.translate(matTranslate, matTranslate, vec3.fromValues(.4, -.4, 0))
mat4.rotateZ(matTranslate, matTranslate, .15 * TAU)
```

First, create an identity matrix. Then, left multiply a translation matrix. Finally, left multiply a rotation matrix about Z.

Wait, I thought we wanted to rotate first!

Yup, that's what we're doing. The result is `I*T*R`. Calling these methods results in a right-multiplication, so the last one happens first.

Making a matrix (4)

The first argument most of the static matrix methods is the output matrix.

`gl-matrix` prefers to write its result directly into the matrix instead of returning it. This is much faster (e.g., rotation only requires setting 4 elements instead of allocating and initializing a new 16 element matrix).

The second argument is the matrix to be transformed.

The third argument is how it will be transformed.

So `mat4.rotateZ(matTranslate, matTranslate, .15 * TAU)` means "rotate `matTranslate` by a 15% turn; write the result to `matTranslate`.

Using a matrix in a shader

Here's an example of how a matrix gets used in a shader:

```
@group(0) @binding(0) var<uniform> model: mat4x4<f32>;
// the matrix is always attached to a bind group
struct VertexOutput {
    @builtin(position) pos: vec4f,
    @location(0) color: vec4f,
};

@vertex fn vs(@location(0) pos: vec3f) ->  VertexOutput {
    var vo: VertexOutput;
    vo.pos = model * vec4(pos, 1); // multiply here
    vo.color = vec4(1, 0, 0, 1);
    return vo;
}
```

Uniforms

Notice that we said `var<uniform>` when declaring the matrix.

A uniform is a piece of data that is read-only inside the shader, but is allowed to change between draw calls.

Textures are a form of uniform data, but they live in their own address space, so we don't write `var<uniform>` for them.

If you're curious, the alternative to `var<uniform>` is `var<storage>`, which is a variable that the shader is allowed to write to.

We don't want to write to the matrix, we want to multiply by it, which we do. We multiply every vertex by the given "model" matrix.

Model matrix

The model matrix is the first time we're seeing a matrix in the shader, but it won't be the last.

The purpose of the model matrix is to transform the 3D model.

It can also be called the "world" matrix, because it takes us from model coordinates (where 0 is the middle of the model) to world coordinates (where 0 is the middle of the scene).

Storing matrices

The matrices in `gl-matrix` are stored as a giant tuple of 16 floats.

They are in row-major order, meaning the first 4 floats is the first row of the matrix, the second 4 is the second row, etc.

In order to send them to the video card, we need to create a GPU buffer, just like we've been using for positions.

However, after we do that, we need to make a bind group.

Bind groups

We've already been introduced to bind groups because we needed them to load shaders.

However, if there's time, let's go over `sample06` together.

I loaded the matrix into a bindgroup along with the color of the triangle (because I wanted to use the same color for the whole triangle).

Then, I changed the viewport 4 times, along with the bindgroup, to draw 4 triangles to different corners of the screen. Each with a different matrix.

Matrix usage summary

- In your shader, declare the matrix as a `var<uniform>`, and give it a binding and a group number, just like for textures.
- We create an identity matrix with `mat4.create()`
- We can transform it however we want using the `mat4...` methods, such as `mat4.translate` or `mat4.rotateZ`. There is a method for every affine matrix except for “shear”.
- A 4x4 matrix is stored as an array of 16 floats.

Matrix usage summary (2)

- To send a matrix to the video card, first create a buffer. Create a `Float32Array` just like in the sample over the mapped range, like we've been doing for vertex buffers.
- Then create a bind group layout. It will have the `buffer = {}` field.
- Create a bind group, using the layout you created. Have it point to the buffer you put your matrix in.
- Make sure your pipeline also has that bind group layout in it.
- When defining your render pass, set the corresponding `@group` to the bind group you created.

Warning about odd sizes

Your GPU secretly stores all vectors with 4 elements.

So even if you have a `vec2f`, it is secretly using the space of a `vec4f`.

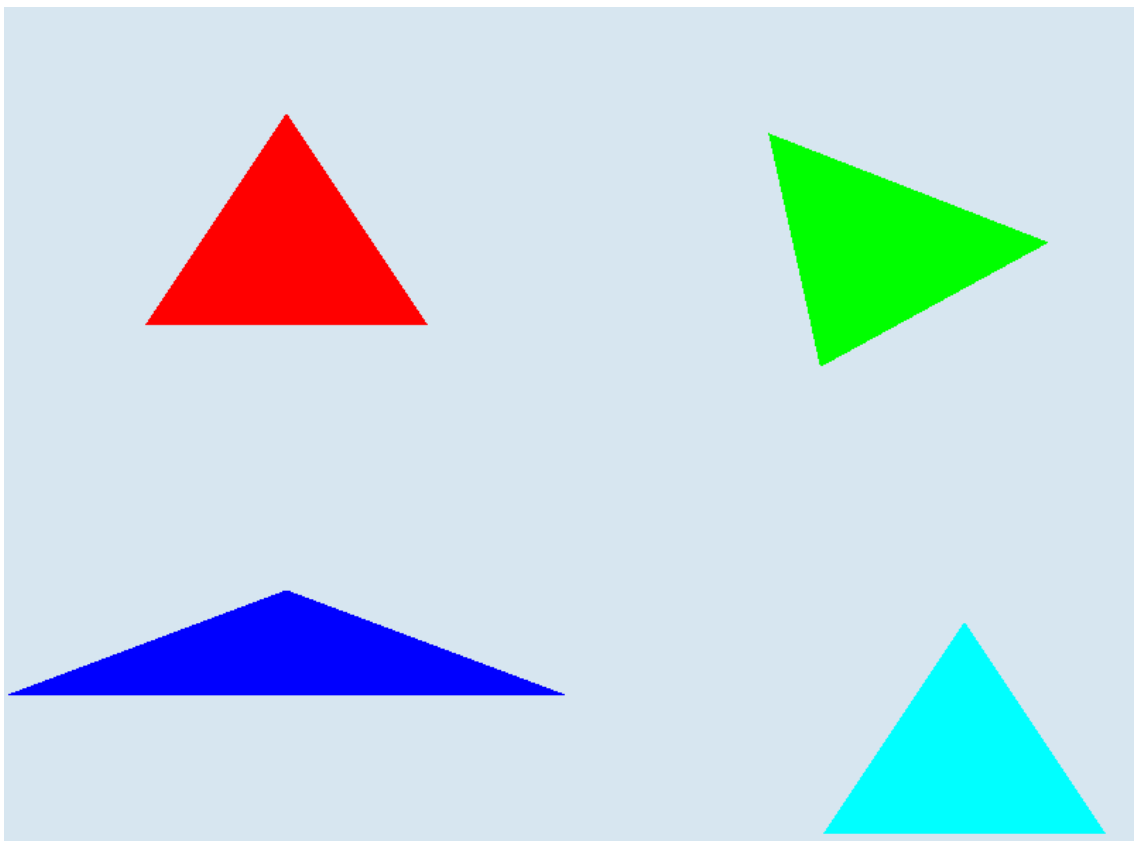
This is also true of matrices: each row receives an entire vector.

So if you create a `mat3x3`, you need to be sure that when you load it, you pad out each row. You'll store it in a vertex array as if it were a 3-by-4 matrix.

If you don't do this, every 4th element will get dropped. I did this when I was coding my initial sample for the lighting chapter. It was rough.

I recommend just using 4x4 matrices for now.

Sample screenshot



Homework

Make sure you can download and run `sample06`.

Tweak the matrices to make the triangles move around.

Make sure to review how bind-groups work. We need them for matrices!

Work through the book chapter.

Just to make it explicit, it is imperative to review the samples. Consider it homework to always review a sample which is mentioned in a lecture. They have important details that you will need to do projects (such as needing to use `GPUBufferUsage.UNIFORM` for buffers used in uniforms).